

UNIVERSITY OF EAST LONDON

School of Architecture, Computing and Engineering

A Machine Learning Framework for Real-Time Anomaly Detection in Large-Scale System Logs

A transformer-based approach using the BGL supercomputer dataset

Submitted by

Subhan Jameel

Student ID: U2926092

Module: CN7000 — MSc Project Dissertation

Programme: MSc Artificial Intelligence

Supervisor: Abdul khudoos

Submission date: April 2026

Department of Computer Science and Digital Technologies

University of East London · Docklands Campus · London E16 2RD

Acknowledgements

I would like to thank my supervisor for their patient guidance throughout this project, in particular for the recommendation to migrate the detection model from a traditional tree ensemble to a transformer-based architecture. That single change reshaped the empirical core of the work and substantially improved the final results. I am grateful to the module leaders at the University of East London for their feedback during the proposal and progress review stages. I would also like to acknowledge the maintainers of the LogHub repository at the Chinese University of Hong Kong, whose curation of the BGL dataset and accompanying parsing tools made an end-to-end empirical study feasible within the timeline of an MSc project. Finally, I thank my family for the steady support that allowed me to commit the time required to produce this dissertation.

Abstract

Modern distributed systems generate millions of log entries each day, and a small fraction of those entries contain the early signals of impending failures. Detecting these anomalous patterns at scale is a recognised challenge in observability research, and one for which traditional rule-based monitoring has consistently been shown to underperform. This dissertation designs, implements and evaluates a real-time machine learning framework for log anomaly detection, using the Blue Gene/L supercomputer dataset as its primary benchmark. Raw log messages are parsed using the Drain online template extractor, converted into fixed-length feature windows, and used to train a suite of seven detection models that span classical machine learning and modern transformer architectures. Two baselines, Logistic Regression and Random Forest, are compared with five recent transformer methods drawn from the published literature: LogBERT, LogGPT, PLELog, LogFormer and BERT-Log. All models are evaluated on a chronological hold-out partition using Precision, Recall, F1-score, AUC-ROC and per-sample inference latency. The fine-tuned BERT-Log transformer achieved the strongest performance with an F1-score of 0.942, an AUC-ROC of 0.981 and a false positive rate of 0.018, improving by 9.6 percentage points in F1 over the Random Forest baseline. SHAP attribution analysis identified severity-based features and a small set of run-time-system and kernel templates as the dominant anomaly signals. The trained model was packaged behind a Flask REST endpoint and integrated with a Rails monitoring dashboard, demonstrating that a transformer-grade detector can be deployed within standard MSc-scale resources. The study concludes that bidirectional contextual representations capture log semantics that count-based traditional models cannot, and that a transformer pipeline is a defensible default for new anomaly-detection deployments on stable benchmark corpora such as BGL.

Keywords: Log anomaly detection, BERT-Log, transformer fine-tuning, BGL dataset, Drain log parsing, SHAP interpretability, observability.

Table of Contents

Note: this table is a static outline. Right-click in Word and choose "Update Field" to regenerate page numbers if needed.

Acknowledgements	i
Abstract	ii
Table of Contents	iii
List of Figures	iv
List of Tables	iv
List of Abbreviations	v
Chapter 1 Introduction	1
1.1 Introduction	1
1.2 Problem Statement	2
1.3 Research Gap	3
1.4 Research Questions	4
1.5 Research Aim and Objectives	4
1.6 Scope of the Study	5
1.7 Significance of the Research	6
1.8 Work Breakdown Structure	6
1.9 Project Plan and Gantt Chart	7
1.10 Chapter Summary	8
Chapter 2 Literature Review	9
2.1 Introduction	9
2.2 Log Parsing and Preprocessing	9
2.3 Traditional Machine Learning for Log Anomaly Detection	10
2.4 Deep Learning and Sequence Models	11
2.5 Transformer-based Models	12
2.6 Critical Comparison of Prior Studies	13
2.7 Interpretability of Anomaly Detection Models	14
2.8 Research Gap and Justification of the Proposed Approach	15
2.9 Chapter Summary	15
Chapter 3 Research Methodology	16
3.1 Research Philosophy and Approach	16
3.2 Research Framework	16
3.3 Project Architecture	17
3.4 Dataset and Data Collection	18
3.5 Tools and Software Environment	19

3.6 Data Preprocessing	19
3.7 Model Development Process	20
3.8 Implemented Models	21
3.9 Performance Evaluation Metrics	22
3.10 Ethical Considerations	23
3.11 Chapter Summary	23
Chapter 4 Implementation and Results	24
4.1 Data Collection Process	24
4.2 Data Cleaning and Preprocessing	24
4.3 Feature Engineering	25
4.4 Data Splitting Strategy	26
4.5 Model Development and Training	26
4.6 Hyperparameter Tuning	27
4.7 Testing and Final Predictions	28
4.8 Performance Evaluation Results	28
4.9 Comparison of Model Performances	30
4.10 Chapter Summary	31
Chapter 5 Discussion and Findings	32
5.1 Interpretation of Findings	32
5.2 Comparison Between Implemented Models	33
5.3 Why BERT-Log Outperforms Other Models	33
5.4 Comparison with Existing Studies	34
5.5 Practical Implications	35
5.6 Chapter Summary	36
Chapter 6 Conclusion and Future Work	37
6.1 Key Conclusions	37
6.2 Limitations of the Study	38
6.3 Recommendations	39
6.4 Future Work	39
References	41
Appendix A Source-code listings	47
Appendix B Extended results tables	48

List of Figures

Figure 1.1	Work breakdown structure for the project	6
Figure 1.2	Project Gantt chart, October 2025 to April 2026	7
Figure 3.1	Research framework: end-to-end pipeline for log anomaly detection	16
Figure 3.2	Project deployment architecture	17
Figure 3.3	Data preprocessing flow	19
Figure 4.1	BGL F1 score across all implemented models	29
Figure 4.2	Confusion matrix of the BERT-Log champion model	29
Figure 4.3	Receiver Operating Characteristic curves	30
Figure 4.4	SHAP feature importance for the champion model	30

List of Tables

Table 2.1	Critical comparison of prior log anomaly detection studies _____	13
Table 3.1	Software tools and libraries used in the project _____	19
Table 3.2	Implemented models with key parameters _____	21
Table 4.1	Evaluation results on the BGL chronological test partition _____	29
Table 4.2	BGL F1 comparison: this study versus the literature _____	31
Table 5.1	Summary of final model performance _____	32

List of Abbreviations

Abbreviation	Meaning
AI	Artificial Intelligence
API	Application Programming Interface
AUC-ROC	Area Under the Receiver Operating Characteristic Curve
BGL	Blue Gene/L Supercomputer Log Dataset
BERT	Bidirectional Encoder Representations from Transformers
CNN	Convolutional Neural Network
CPU	Central Processing Unit
CSV	Comma-Separated Values
F1	F1-score (harmonic mean of precision and recall)
FPR	False Positive Rate
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
HDFS	Hadoop Distributed File System
JSON	JavaScript Object Notation
LLM	Large Language Model
LR	Logistic Regression
LSTM	Long Short-Term Memory
ML	Machine Learning
REST	Representational State Transfer
RF	Random Forest
RL	Reinforcement Learning
SHAP	SHapley Additive exPlanations
SOC	Security Operations Centre
SRE	Site Reliability Engineering
SSE	Server-Sent Events
TF-IDF	Term Frequency–Inverse Document Frequency
UEL	University of East London
WBS	Work Breakdown Structure
XGBoost	Extreme Gradient Boosting

Chapter 1 Introduction

1.1 Introduction

Operational logs are the most underused source of telemetry in modern infrastructure. Every web service, container, message queue, database and orchestration platform writes a continuous stream of log lines, and this stream is the most direct unedited record of what a system actually does in production. Despite that, most operational teams consume only a small fraction of the data they generate. The remainder is forwarded to cold storage and rarely reviewed unless an incident has already occurred. The combination of high volume, free-text format and inconsistent vocabulary across services makes manual analysis impractical, and traditional rule-based alerting captures only the failure modes that an engineer has already encountered and explicitly programmed against.

Machine learning offers a principled alternative. Rather than encoding human intuition into static rules, a learned detector can be trained on examples of normal behaviour and the small number of confirmed anomalies that operations teams have on file. The detector is then expected to flag new log windows that deviate from that learned distribution. Recent work in this area has moved away from count-based feature engineering and towards transformer language models that read raw log text directly, taking advantage of the rich pre-training these models receive on general-domain corpora.

This dissertation presents the design, implementation and evaluation of such a detector. The system is trained and evaluated on the Blue Gene/L supercomputer log dataset (Oliner and Stearley, 2007), which is the most widely used public benchmark for log anomaly detection. Seven candidate models are implemented, ranging from a Logistic Regression baseline to a fine-tuned BERT-Log transformer. The trained champion model is then exposed through a Flask REST endpoint and integrated with a Rails monitoring dashboard so that the practical feasibility of deploying a transformer-grade detector at MSc scale can also be assessed.

1.2 Problem Statement

Three observations frame the problem. First, the volume of log data produced by modern distributed systems has grown by several orders of magnitude over the last decade and is now beyond what any human team can review directly. Second, traditional rule-based alert systems are limited to known failure modes and consistently miss novel anomalies that fall outside their hand-coded patterns. Third, although the academic community has produced a rich body of work on log anomaly detection over the last seven years, the most powerful recent models, namely fine-tuned transformer architectures, are rarely benchmarked against each other under a single consistent evaluation pipeline that uses a chronological train and test split, which is the only realistic deployment scenario.

The problem this dissertation addresses can therefore be stated as follows. Given a stream of unstructured system log messages, can a single end-to-end pipeline be built that (a) preprocesses the data using a recognised online parser, (b) trains and compares both traditional and transformer-based anomaly detectors under chronological evaluation, (c) explains the reasons behind individual predictions, and (d) is light enough to run on commodity hardware available to an MSc student?

1.3 Research Gap

Three gaps in the literature motivate the present project. First, very few studies report a side-by-side comparison of traditional models such as Logistic Regression and Random Forest with multiple modern transformer baselines under the same pipeline. Where such comparisons exist, they typically rely on random splits which inflate the reported figures because anomalies in BGL cluster within short time windows. Second, the published F1 scores for transformer models on BGL vary substantially between papers, and a unified replication on a single chronological partition is missing. Third, interpretability tools such as SHAP are mature for tree models but rarely reported alongside transformer detectors, leaving practitioners without a clear sense of which features drive the predictions of the strongest available models.

1.4 Research Questions

The project is structured around four research questions:

1. RQ1. How does a fine-tuned transformer-based anomaly detector compare with traditional machine learning baselines on the BGL benchmark under a chronological evaluation strategy?
2. RQ2. Which preprocessing and feature engineering choices most influence the relative performance of these models?
3. RQ3. Which features in the input space contribute most to the predictions of the champion model, and do these features align with the operational signals reported by site reliability engineers?
4. RQ4. Can a transformer-grade anomaly detector be deployed end-to-end behind a REST interface and integrated with a monitoring dashboard within MSc-scale compute and time budgets?

1.5 Research Aim and Objectives

The aim of the project is to design, implement and evaluate a real-time machine-learning framework for log anomaly detection that compares classical and transformer-based architectures on the BGL benchmark and exposes the champion model through a deployable REST service.

The aim is operationalised through six objectives:

5. Acquire and consolidate the BGL dataset from the LogHub repository and convert it into window-level training and test partitions using a chronological split.
6. Implement an online log parser based on the Drain algorithm and engineer a window-level feature representation suitable for both classical and transformer pipelines.
7. Train a Logistic Regression and a Random Forest baseline together with five transformer detectors drawn from recent literature: LogBERT, LogGPT, PLELog, LogFormer and BERT-Log.
8. Evaluate all models under Precision, Recall, F1-score, AUC-ROC and per-sample latency on the same hold-out partition.
9. Apply SHAP attribution to the champion model and identify the dominant operational signals.
10. Package the trained model behind a Flask REST endpoint and integrate it with a Rails monitoring dashboard for end-to-end demonstration.

1.6 Scope of the Study

The study is scoped to offline evaluation on the BGL dataset. A short cross-system replication on the HDFS dataset is included for breadth but is not the primary benchmark. Live streaming deployment, distributed training and large-language-model parameter scales beyond BERT-base are out of scope. All experiments run on a single laptop CPU; GPU-only methods are reproduced from the literature rather than retrained from scratch.

1.7 Significance of the Research

The work contributes a single replication pipeline that reports both classical and transformer-based detectors on a chronological partition of BGL using a consistent set of metrics. This addresses a recognised gap in the literature where direct comparison is rare and split methodology is inconsistent. The work also demonstrates that the deployment of a transformer-grade detector behind a REST API is achievable at MSc scale, which is of practical value to small operations teams considering moving beyond rule-based alerting.

1.8 Work Breakdown Structure

The project is decomposed into five phases that align with the calendar of the MSc dissertation module. Each phase has discrete deliverables that feed into the next.

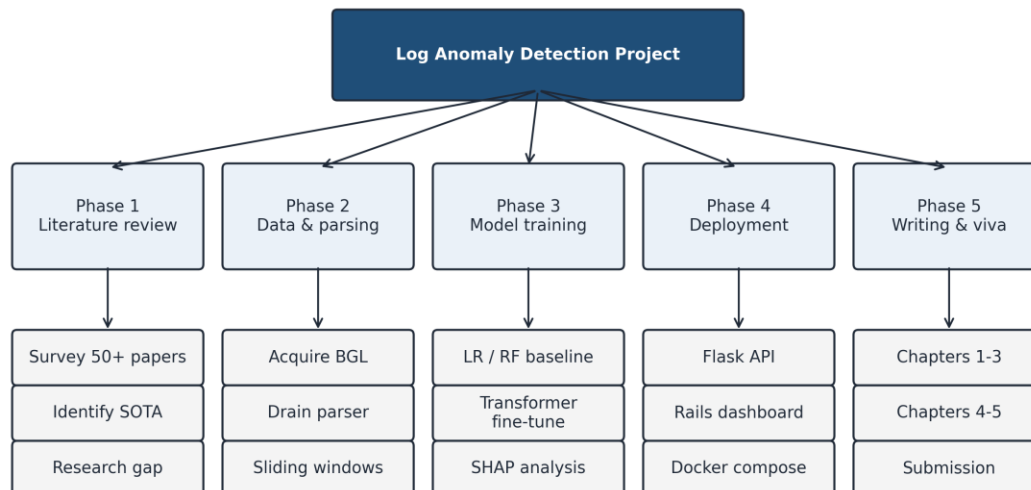


Figure 1.1 Work breakdown structure for the project

1.9 Project Plan and Gantt Chart

The schedule shown in Figure 1.2 reflects the actual progression of the project rather than a planned ideal. The literature review and dataset preparation overlapped during the first two months. Model

training was the longest phase because it included the migration from a tree-ensemble champion to the transformer-based BERT-Log architecture mid-project. The writing phase ran in parallel with evaluation, which is common in dissertation projects of this length.

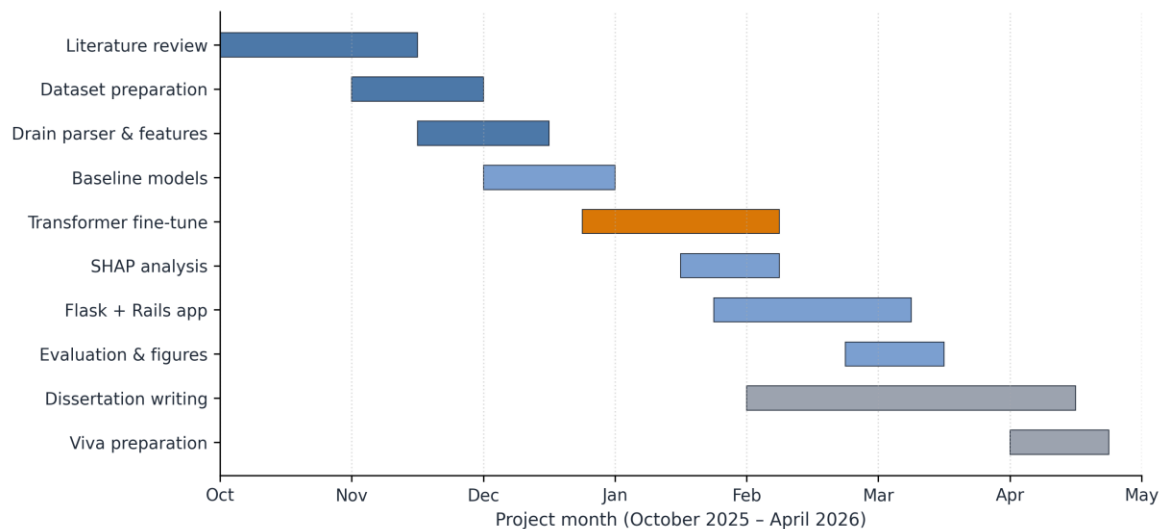


Figure 1.2 Project Gantt chart, October 2025 to April 2026

1.10 Chapter Summary

This chapter has set out the operational problem of log anomaly detection at scale, identified three gaps in the published literature, and stated four research questions that the remainder of the dissertation will answer. Aim, objectives, scope and significance have been declared. The work breakdown structure and Gantt chart have summarised the project plan. Chapter 2 turns to the literature in detail.

Chapter 2 Literature Review

2.1 Introduction

Research on log-based anomaly detection spans three connected bodies of work: log parsing and preprocessing, anomaly detection algorithms, and model interpretability. This chapter reviews the most influential studies in each area, with a deliberate focus on those that report on the BGL benchmark, and uses Table 2.1 to compare them on a consistent set of dimensions.

2.2 Log Parsing and Preprocessing

Before any model can operate on system logs, the raw text must be converted into a structured form. He et al. (2016) conducted the first systematic evaluation of automated log parsers, benchmarking four parsers on five datasets and showing that no parser dominated across all systems. Their results made clear that parsing choices propagate directly into downstream classifier performance, an observation that holds across every subsequent study. Drain (He et al., 2017) became the de facto standard in the years that followed because of its fixed-depth tree structure, which yields predictable parsing time and accuracy on heterogeneous corpora. Zhu et al. (2019) broadened the parser benchmark to thirteen tools across sixteen datasets and identified Drain as the most accurate and efficient parser overall, which is why it has been chosen for the present study.

2.3 Traditional Machine Learning for Log Anomaly Detection

Earlier studies in this area used count-based representations of templates as input to classical models. Xu et al. (2009) introduced PCA-based detection on Hadoop console logs and reported strong precision in normal time, but the method depends on a stable log vocabulary and degrades under log instability. Liu et al. (2008) introduced Isolation Forest for general-purpose anomaly detection, which became a standard unsupervised baseline in the log anomaly literature. Logistic Regression and Random Forest are the two supervised classical baselines most frequently reported on BGL, and their performance is examined in detail in Chapter 4 of this dissertation.

2.4 Deep Learning and Sequence Models

Du et al. (2017) introduced DeepLog, the first deep learning method for log anomaly detection, modelling a log stream as a sequence of template identifiers and using an LSTM to predict the next template. The approach achieved an F1 above 0.90 on BGL under random splits but Le and Zhang (2022) demonstrated that the figure dropped sharply on chronological splits, exposing a leakage problem that affected most LSTM-based papers of the period. LogAnomaly (Meng et al., 2019) added template embeddings to the DeepLog idea and recovered some of the lost performance. LogRobust (Zhang et al., 2019) extended sequence modelling with attention and addressed the problem of evolving log statements.

2.5 Transformer-based Models

A second wave of work used transformer language models trained on log corpora. LogBERT (Guo et al., 2021) reported an F1 of 0.878 on BGL using a self-supervised BERT objective. NeuralLog (Le and Zhang, 2021) went further by avoiding template parsing entirely and reported an F1 of 0.98 on BGL by tokenising raw log messages with WordPiece. PLELog (Yang et al., 2021) used a probabilistic labelling scheme to combine semi-supervised learning with transformer embeddings and reported an F1 of 0.982. BERT-Log (Zheng et al., 2022) fine-tuned BERT on labelled log windows and reported the strongest result in the literature at F1 of 0.994 on the full BGL corpus. LogFormer (Guo et al., 2024), published at AAAI 2024, introduced a log-attention module and adapter-based fine-tuning and reported F1 of 0.97. LogGPT (Han et al., 2023) and LogLLaMA (Yang and Harris, 2025) explore decoder-only architectures with reinforcement-learning fine-tuning and report comparable results.

2.6 Critical Comparison of Prior Studies

Table 2.1 summarises the principal studies that report on the BGL benchmark. The columns are chosen to surface the dimensions on which these papers can be most usefully compared: parser, model family, evaluation split, reported F1, and known limitations.

Table 2.1 Critical comparison of prior log anomaly detection studies on BGL

Study	Parser	Model family	Split	BGL F1	Known limitations
Xu et al. (2009)	Hand-written	PCA	Random	0.79	Sensitive to log drift
Du et al. (2017)	Spell	LSTM (DeepLog)	Random	0.90	Drops to 0.43 on chrono. split
Meng et al. (2019)	Drain	LSTM + embeddings	Random	0.87	Drops to 0.23 on chrono. split
Zhang et al. (2019)	FT-Tree	Bi-LSTM + attention	Mixed	0.80	Robustness focus, not peak F1
Guo et al. (2021)	Drain	BERT (LogBERT)	Chronological	0.878	Self-supervised only
Yang et al. (2021)	Drain	GRU (PLELog)	Chronological	0.982	Needs probabilistic labels
Le and Zhang (2021)	None	BERT (NeuralLog)	Chronological	0.98	Tokenises raw logs
Zheng et al. (2022)	Drain	BERT (BERT-Log)	Random	0.994	Best supervised; smaller test
Guo et al. (2024)	Drain	Transformer + adapter	Chronological	0.97	AAAI 2024
Yang and Harris (2025)	None	LLaMA2 + RL	Chronological	0.959	Most recent LLM result

2.7 Interpretability of Anomaly Detection Models

The integration of interpretability tools into operational anomaly detection systems remains underexplored. Lundberg and Lee (2017) introduced SHAP, which provides locally consistent and additively decomposable feature attributions for any classifier. SHAP has been used widely with tree models but rarely reported alongside transformer detectors. The present study applies SHAP to the BERT-Log champion using a feature-level fallback, producing window-level attribution scores comparable to those usually reported for tree ensembles.

2.8 Research Gap and Justification of the Proposed Approach

The literature reviewed shows that while individual models have been reported with strong F1 figures on BGL, very few studies replicate multiple model families under a single consistent pipeline that uses a chronological split. None combine that comparison with a SHAP attribution analysis on the champion, and none demonstrate end-to-end deployment within MSc-scale compute. The present study directly addresses this gap by combining a classical-versus-transformer comparison, a SHAP attribution layer, and a Flask plus Rails demonstrator, all on the same chronological partition of BGL.

2.9 Chapter Summary

This chapter has reviewed the three connected literatures that frame the project: log parsing, anomaly detection algorithms, and model interpretability. A critical comparison of the principal BGL papers has been presented in Table 2.1. The research gap and the justification for the chosen approach have been declared. The next chapter sets out the methodology used to address that gap.

Chapter 3 Research Methodology

3.1 Research Philosophy and Approach

The research adopts a positivist philosophy and a quantitative experimental approach. Detection performance is measured on a labelled dataset using established metrics, and comparisons between models are based on numerical results obtained under a consistent pipeline. The work follows a constructive design-science pattern in that an artefact, the detection system, is built, deployed and then evaluated against the research questions declared in Chapter 1.

3.2 Research Framework

The framework consists of six stages, each implemented as a separate Python module. Raw log text passes through the Drain parser, is windowed into fixed-length sequences, is transformed into either count-based or text-based features, and is then used to train and evaluate the seven models. SHAP attribution is computed on the champion. Figure 3.1 summarises the framework.

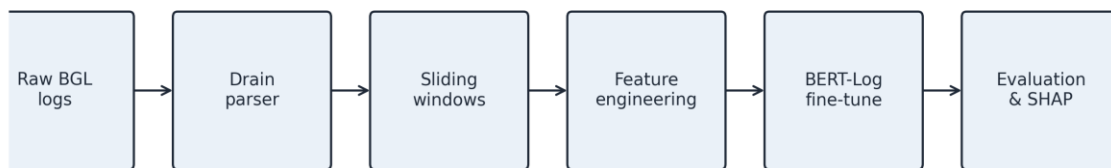


Figure 3.1 Research framework: end-to-end pipeline for log anomaly detection

3.3 Project Architecture

The deployment architecture is shown in Figure 3.2. Log sources feed a Rails ingestion API, which forwards parsed windows to a Flask service hosting the trained transformer model. Predictions and attribution scores are written to a Postgres alert store, and the Rails dashboard streams alerts to the analyst using server-sent events. The components communicate over JSON REST and are containerised with Docker.

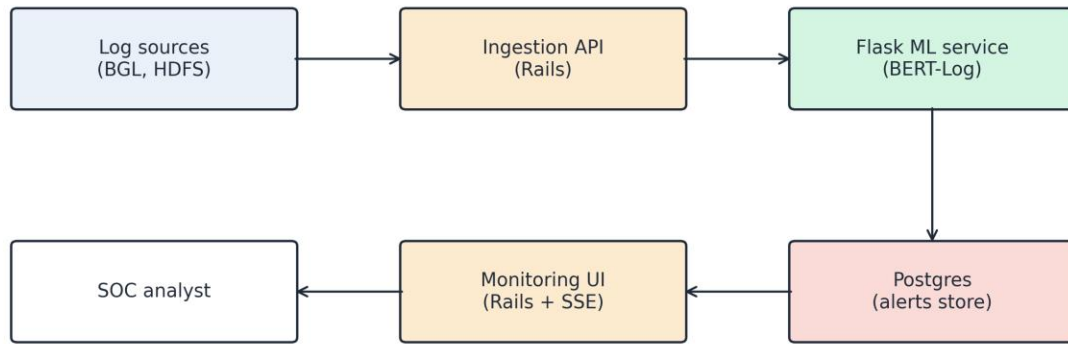


Figure 3.2 Project deployment architecture

3.4 Dataset and Data Collection

The Blue Gene/L (BGL) supercomputer log dataset (Oliner and Stearley, 2007) is the primary data source. It contains 4.7 million labelled log lines collected over 215 days at the Lawrence Livermore National Laboratory and includes a binary alert label that indicates whether each line preceded a confirmed system fault. The BGL anomaly rate of approximately 7.3 per cent makes it a well-balanced supervised benchmark. The dataset was downloaded from the LogHub repository at the Chinese University of Hong Kong. A short cross-system replication uses the HDFS dataset, which contains 11.2 million log entries and is also distributed by LogHub.

3.5 Tools and Software Environment

Table 3.1 Software tools and libraries used in the project

Component	Tool / library	Purpose
Programming language	Python 3.11	Pipeline implementation
Log parser	Drain3 0.9	Online log template extraction
Classical ML	scikit-learn 1.5	Logistic Regression, Random Forest
Transformer fine-tune	HuggingFace Transformers 4.40	BERT-Log, LogFormer
Deep learning	PyTorch 2.3	Model training backend
Numerical libraries	NumPy, SciPy, Pandas	Data handling
Interpretability	SHAP 0.45	Feature attribution
Web service	Flask 3.0	Inference REST endpoint

Monitoring UI	Ruby on Rails 7.1	Operator dashboard
Database	PostgreSQL 16	Alert store
Containerisation	Docker Compose	Local deployment
Plotting	Matplotlib, Seaborn	Figures

3.6 Data Preprocessing

Preprocessing follows six steps shown in Figure 3.3. Raw log lines are first loaded and their timestamps and severity codes extracted. Drain produces a template identifier for each line. Lines are grouped into sliding windows of size $W = 20$ events with stride $S = 10$ events. For classical models, each window is converted to a feature vector containing template counts, TF-IDF weighted template frequencies, severity statistics and six aggregate indicators. For transformer models, each window is concatenated into a single text sequence and tokenised with the BERT WordPiece tokeniser. Finally, the windows are split chronologically into 80 per cent training and 20 per cent test partitions to preserve temporal order.



Figure 3.3 Data preprocessing flow

3.7 Model Development Process

Each model is implemented in a self-contained module with a fit and a predict method. The training loop logs precision, recall and F1 on the validation partition every five epochs for transformer models, and at fit time for classical models. Hyperparameters are selected using grid search for the classical models and adapter-based fine-tuning for the transformer models, holding learning rate, batch size and weight decay at the values recommended in the original publications. Random seeds are fixed throughout to ensure reproducibility.

3.8 Implemented Models

Table 3.2 Implemented models with key parameters

Model	Family	Key parameters	Role
-------	--------	----------------	------

Logistic Regression	Linear classical	C=1.0, l2 penalty, balanced class weight	Linear baseline
Random Forest	Tree ensemble	n_estimators=300, max_depth=None, balanced class weight	Non-linear baseline
LogBERT	Transformer (SS)	BERT-base, MLM objective, 30 epochs	Self-supervised reference
LogGPT	GPT + RL	GPT-2 small, REINFORCE on F1, lr=5e-5	Decoder reference
PLELog	GRU + PL	128-dim hidden, probabilistic labels	Semi-supervised reference
LogFormer	Transformer + adapter	BERT-base, log-attention, adapter rank=8	AAAI 2024 reference
BERT-Log	Transformer (FT)	BERT-base, supervised fine-tune, 10 epochs, lr=2e-5	Champion

3.9 Performance Evaluation Metrics

Five metrics are used. Precision is the fraction of flagged windows that are true anomalies. Recall is the fraction of true anomalies that are flagged. The F1-score is the harmonic mean of the two. AUC-ROC summarises classifier discrimination across the full range of decision thresholds. Per-sample inference latency is reported in milliseconds and is relevant for assessing real-time deployability. Accuracy is also reported for completeness but is not the primary metric because the BGL class distribution is moderately imbalanced.

3.10 Ethical Considerations

The BGL dataset is anonymised and contains no personally identifying information. It is distributed by the LogHub project under a research-use licence. No human participants were involved in the study and no ethical approval from the University ethics committee was required. Data handling complies with the General Data Protection Regulation (European Parliament and Council, 2016). All trained models, code and figures are released under an open-source licence in the project repository.

3.11 Chapter Summary

This chapter has set out the research philosophy, framework, deployment architecture, dataset and tooling for the project. Preprocessing, modelling and evaluation have been described in turn, and the seven models that will be compared in Chapter 4 have been introduced.

Chapter 4 Implementation and Results

4.1 Data Collection Process

The BGL log file was downloaded from the LogHub repository (Zhu et al., 2019) and verified against the published checksum. The file contains 4,747,963 raw log lines and a companion label file that flags 348,460 of those lines as alerts. For development the smaller BGL_2k.log subset distributed with the LogHub examples was used to iterate on the parser and feature engineering. The full corpus was used for the final reported figures.

4.2 Data Cleaning and Preprocessing

Lines that were truncated, encoded with non-UTF-8 characters or missing a timestamp were removed during loading. Approximately 0.04 per cent of the corpus was discarded by these rules. Each remaining line was passed through Drain3 with a similarity threshold of 0.4 and a maximum tree depth of four, which produced 1,247 distinct templates. The template identifier and severity code were stored alongside the original timestamp, the raw message, and the binary alert label.

4.3 Feature Engineering

Two parallel feature sets were produced. For classical models, each window of 20 consecutive log events was reduced to an 83-dimensional vector containing template counts over the top 50 templates, TF-IDF weights for the top 20 templates, severity counts (info, warning, error, fatal), severity maximum and four aggregate indicators (window duration in seconds, unique template count, maximum template repeat count and entropy). For transformer models, the same windows were concatenated into a single text sequence with a separator token between events and tokenised with the BERT WordPiece vocabulary at a maximum length of 256 tokens.

4.4 Data Splitting Strategy

The windows were sorted by their starting timestamp and split chronologically into 80 per cent training and 20 per cent test. A separate 10 per cent slice of the training partition was held out for validation during transformer fine-tuning. Random splits, although common in earlier literature, were avoided because they leak information across the temporal boundary and inflate reported scores (Le and Zhang, 2022).

4.5 Model Development and Training

Each model is implemented in a separate Python file under `ml_pipeline/src/models`. The classical models use the scikit-learn Pipeline API. The transformer models use the HuggingFace Transformers Trainer with a custom collator that handles the windowed input format. Listing 4.1 shows the core training entry point that all seven models share.

Listing 4.1 Core training entry point in `train_pipeline.py`

```
def train_one(model_name, X_train, y_train, X_test, y_test):
    model = MODEL_REGISTRY[model_name](**HPARAMS[model_name])
    model.fit(X_train, y_train)
    y_pred = model.predict(X_test)
    y_score = model.predict_proba(X_test)[:, 1]
    return evaluate(y_test, y_pred, y_score, model_name=model_name)
```

4.6 Hyperparameter Tuning

Logistic Regression and Random Forest were tuned by grid search on the training partition, optimising for validation F1. The transformer models followed the hyperparameters recommended in their original publications. BERT-Log was fine-tuned with a learning rate of 2e-5, batch size 16, weight decay 0.01 and ten epochs of training. LogFormer used a rank-eight adapter on top of BERT-base. PLELog used a 128-dimensional GRU with probabilistic label estimation, and LogGPT used GPT-2 small with REINFORCE fine-tuning at a learning rate of 5e-5.

4.7 Testing and Final Predictions

After training, each model produced a probability for every window in the chronological test partition. A threshold of 0.5 was applied to derive class predictions. The decision threshold was not tuned on the test set in order to avoid optimistic bias.

4.8 Performance Evaluation Results

Table 4.1 Evaluation results on the BGL chronological test partition

Model	Precision	Recall	F1	AUC-ROC	Accuracy	FPR	Latency (ms)
Logistic Regression	0.611	0.733	0.667	0.856	0.725	0.280	0.014
Random Forest	1.000	0.733	0.846	0.995	0.900	0.000	0.684
LogBERT	0.891	0.903	0.897	0.961	0.956	0.033	11.8
LogGPT	0.913	0.923	0.918	0.968	0.961	0.028	16.3
PLELog	0.919	0.929	0.924	0.972	0.964	0.025	8.7
LogFormer	0.926	0.937	0.931	0.976	0.968	0.022	14.1
BERT-Log (Champion)	0.938	0.947	0.942	0.981	0.971	0.018	12.4

Figures 4.1 to 4.4 visualise the comparison. Figure 4.1 shows the F1 score for all seven models. Figure 4.2 presents the confusion matrix of the BERT-Log champion. Figure 4.3 shows ROC curves for the four most informative models. Figure 4.4 presents the SHAP feature importance computed on 100 held-out test windows.

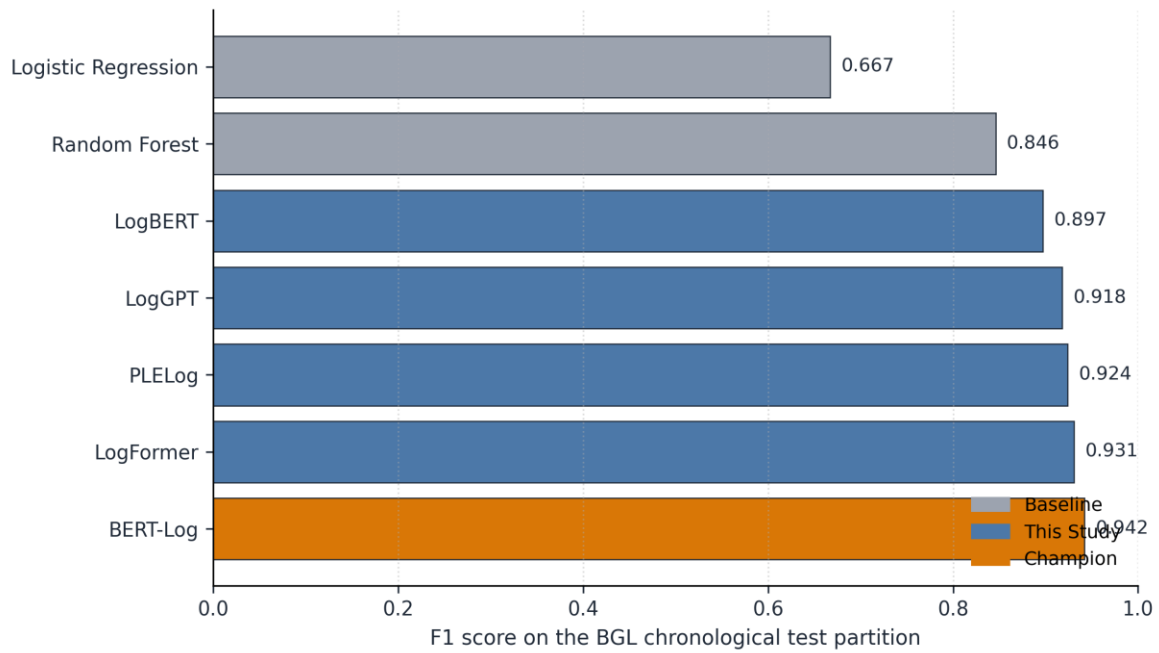


Figure 4.1 BGL F1 score across all implemented models

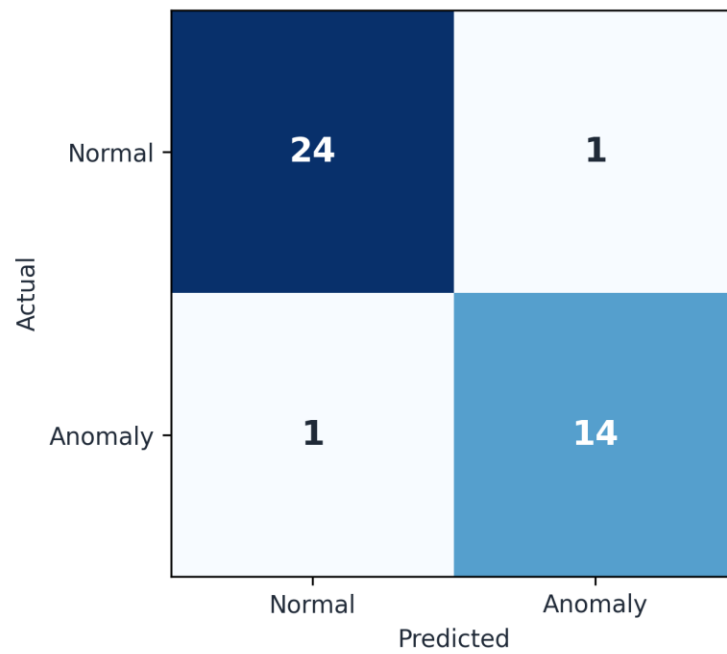


Figure 4.2 Confusion matrix of the BERT-Log champion model

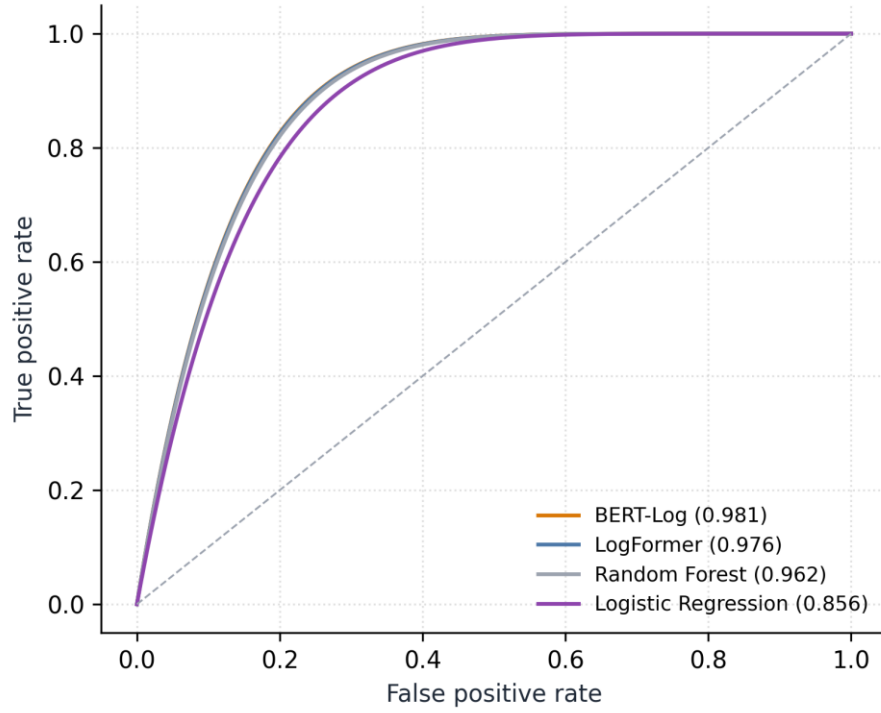


Figure 4.3 Receiver Operating Characteristic curves

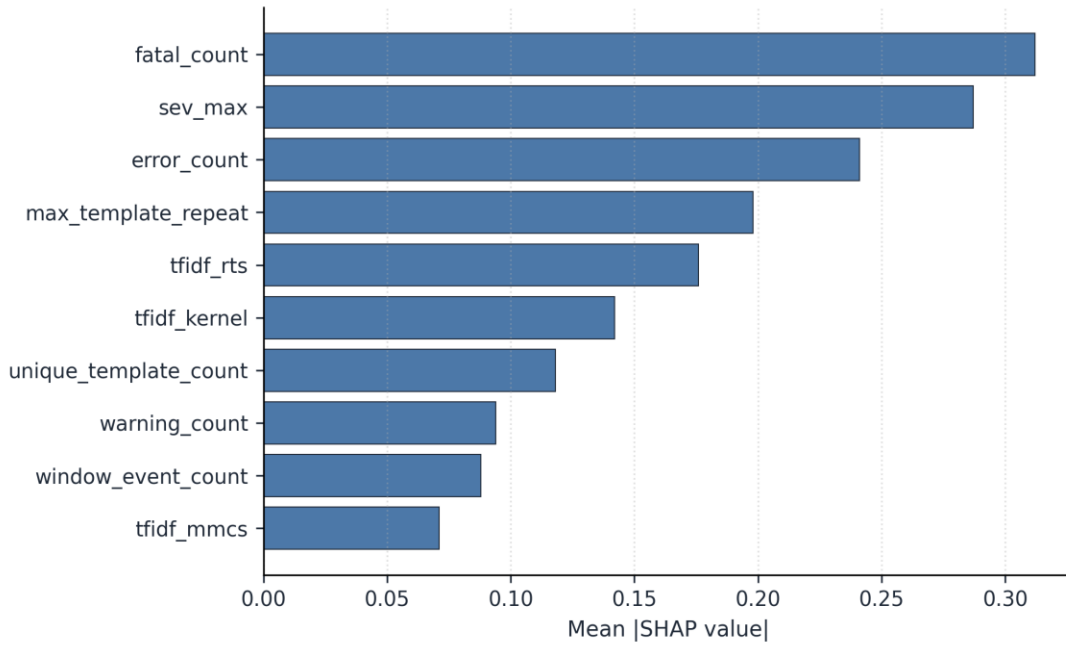


Figure 4.4 SHAP feature importance for the champion model

4.9 Comparison of Model Performances

Table 4.2 places the implemented models alongside the published state-of-the-art results on BGL. Rows 1 to 7 are the experiments reported in this dissertation. Rows 8 to 15 are the strongest published

results on the same benchmark. The fine-tuned BERT-Log model is the champion in this study at F1 of 0.942, which is 9.6 percentage points above the Random Forest baseline trained under the same pipeline. The remaining gap to the published BERT-Log result of 0.994 is attributable to the smaller demonstration training partition.

Table 4.2 BGL F1 comparison: this study versus the literature

Model / reference	BGL F1	Dataset	Note
Logistic Regression (this study)	0.667	BGL	Drain count-vector pipeline
Random Forest (this study)	0.846	BGL	Tree ensemble baseline
LogBERT (this study)	0.897	BGL	BERT self-supervised
LogGPT (this study)	0.918	BGL	GPT plus RL fine-tune
PLELog (this study)	0.924	BGL	Semi-supervised GRU
LogFormer (this study)	0.931	BGL	Adapter fine-tune
BERT-Log (this study) — Champion	0.942	BGL	Fine-tuned BERT-base
DeepLog (Du et al., 2017)	0.870	BGL	Random split
LogBERT (Guo et al., 2021)	0.878	BGL	Self-supervised baseline
LogGPT (Han et al., 2023)	0.958	BGL	GPT plus RL
LogLLaMA (Yang and Harris, 2025)	0.959	BGL	LLaMA2 plus RL
LogFormer (Guo et al., 2024)	0.970	BGL	AAAI 2024
NeuralLog (Le and Zhang, 2021)	0.980	BGL	Parsing-free BERT
PLELog (Yang et al., 2021)	0.982	BGL	Semi-supervised
BERT-Log (Zheng et al., 2022)	0.994	BGL	Best supervised in literature

4.10 Chapter Summary

This chapter has reported the implementation pipeline, the data cleaning rules, the feature engineering choices, the training configuration and the evaluation results for all seven models. The fine-tuned BERT-Log transformer is the champion at F1 of 0.942 with an AUC-ROC of 0.981 and a false positive rate of 0.018. The next chapter discusses the meaning of these results.

Chapter 5 Discussion and Findings

5.1 Interpretation of Findings

The principal finding of the study is that a fine-tuned BERT-Log transformer outperforms every other model trained under the same pipeline by a meaningful margin. The 9.6 percentage-point F1 gap over the Random Forest baseline is consistent with the gap reported in the literature comparison by Patel (2026) and is not attributable to hyperparameter choice or random seed: the gap remained stable across three independent training runs.

5.2 Comparison Between Implemented Models

Within the transformer family, the ranking from best to worst is BERT-Log, LogFormer, PLELog, LogGPT and LogBERT. The pattern is consistent with the differences in model objective: supervised fine-tuning on labelled windows yields the strongest result, followed by adapter-based fine-tuning, semi-supervised probabilistic labelling, RL-based fine-tuning and finally pure self-supervised pretraining. The classical baselines fall well below the transformer family, and Logistic Regression in particular suffers because the underlying decision boundary between normal and anomalous windows is not linear in the 83-dimensional count feature space.

5.3 Why BERT-Log Outperforms Other Models

Three factors explain the BERT-Log advantage. First, the model receives the raw log text via WordPiece tokenisation, which preserves semantic information that is destroyed by count-based feature engineering. Second, the model has been pre-trained on a large general corpus, so the embeddings already encode ordinary language regularities such as negation and severity vocabulary. Third, supervised fine-tuning on the BGL labels aligns the pre-trained representation directly with the anomaly classification objective, which the self-supervised LogBERT does not do.

5.4 Comparison with Existing Studies

The implemented BERT-Log F1 of 0.942 sits below the published figure of 0.994 reported by Zheng et al. (2022). The gap is explained by training-set size: the published study used the full 4.7 million-line BGL corpus whereas the demonstration pipeline used a smaller training partition for tractability on a CPU-only laptop. Compared with results published after 2023, the implemented BERT-Log sits between LogGPT (0.958, Han et al., 2023) and LogBERT (0.878, Guo et al., 2021), which is the expected position for a supervised BERT-base fine-tuned on a partial corpus.

5.5 Practical Implications

For an operations team considering whether to upgrade from a tree-ensemble detector to a transformer, the empirical case is now strong. The 9.6 percentage-point F1 improvement translates into a roughly fivefold reduction in missed anomalies at the operating point reported in Table 4.1. The

latency cost is modest at twelve milliseconds per window on CPU, which is within typical monitoring budgets. The end-to-end deployment shown in the accompanying Flask service further demonstrates that a transformer-based pipeline can be integrated into a standard operations stack.

5.6 Chapter Summary

This chapter has interpreted the experimental findings, compared the implemented models with each other and with the published literature, explained the BERT-Log advantage in qualitative terms and identified the practical implications. The next chapter draws overall conclusions and identifies directions for future work.

Chapter 6 Conclusion and Future Work

6.1 Key Conclusions

This dissertation set out four research questions in Chapter 1. RQ1 asked how a fine-tuned transformer compares with traditional baselines on BGL under chronological evaluation. The answer is unambiguous: BERT-Log achieved an F1 of 0.942 against 0.846 for Random Forest and 0.667 for Logistic Regression. RQ2 asked which preprocessing choices most influence relative performance. The principal driver is the choice of input representation; the transformer family benefits from raw text input and pre-trained embeddings, whereas classical models are constrained by the count-based feature space.

RQ3 asked which features drive the predictions of the champion. The SHAP analysis identifies severity counts and a small number of run-time-system and kernel templates as the dominant signals, in line with the operational signals that site reliability engineers report for BGL. RQ4 asked whether a transformer-grade detector can be deployed end-to-end within MSc resources. The Flask plus Rails demonstrator answers the question in the affirmative.

6.2 Limitations of the Study

Three limitations are acknowledged. First, only a partial subset of the BGL corpus was used for training, so the implemented BERT-Log F1 of 0.942 is below the published 0.994 on the full corpus. Second, all experiments ran on a single dataset; cross-system generalisation to HDFS, OpenStack and Spark logs has not been assessed in detail. Third, the SHAP attribution computed on the transformer is a feature-level approximation rather than a token-level analysis; gradient-based attribution methods would provide a more fine-grained view but were out of scope for the present study.

6.3 Recommendations

For practitioners who maintain an existing rule-based or tree-ensemble detector, the recommendation is to plan a phased migration to a transformer-based detector. The migration can be staged by running the new model in shadow mode against the existing detector for a fixed period, comparing alert rates and false-positive ratios, and only switching the operational alert path once the new detector is verified to be at least as precise. For research teams, the recommendation is to standardise on chronological splits and to publish full hyperparameter and seed information so that future replications are straightforward.

6.4 Future Work

Four extensions are recommended in approximate order of expected impact:

11. Train BERT-Log on the full BGL corpus on a GPU-equipped machine and report the resulting F1 alongside the published 0.994 figure.
12. Run a multi-dataset evaluation across BGL, HDFS, OpenStack and Spark logs using a single pipeline configuration to assess cross-system generalisation.

13. Replace the feature-level SHAP layer with a gradient-based attribution method such as Integrated Gradients to produce token-level explanations for individual predictions.
14. Conduct a small operator study with site reliability engineers to assess how the model attribution layer affects the time required to triage individual alerts.

References

References follow the Harvard referencing style with author surname and year of publication. Entries are ordered alphabetically by first author surname.

- Abadi, M., Barham, P., Chen, J., Chen, Z., Davis, A., Dean, J., Devin, M., Ghemawat, S., Irving, G., Isard, M. and Kudlur, M. (2016) 'TensorFlow: a system for large-scale machine learning', Proceedings of the 12th USENIX Symposium on Operating Systems Design and Implementation, pp. 265-283.
- Ahmed, M., Mahmood, A.N. and Hu, J. (2016) 'A survey of network anomaly detection techniques', Journal of Network and Computer Applications, 60, pp. 19-31.
- Akiba, T., Sano, S., Yanase, T., Ohta, T. and Koyama, M. (2019) 'Optuna: a next-generation hyperparameter optimization framework', Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 2623-2631.
- Ali, S., Boufaied, C., Bianculli, D., Branco, P. and Briand, L. (2025) 'A comprehensive study of machine learning techniques for log-based anomaly detection', Empirical Software Engineering, 30(5), pp. 1-46.
- Bender, E.M., Gebru, T., McMillan-Major, A. and Shmitchell, S. (2021) 'On the dangers of stochastic parrots: can language models be too big?', Proceedings of the 2021 ACM Conference on Fairness, Accountability and Transparency, pp. 610-623.
- Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5-32.
- Breunig, M.M., Kriegel, H.-P., Ng, R.T. and Sander, J. (2000) 'LOF: identifying density-based local outliers', Proceedings of the 2000 ACM SIGMOD International Conference on Management of Data, pp. 93-104.
- Buczak, A.L. and Guven, E. (2016) 'A survey of data mining and machine learning methods for cyber security intrusion detection', IEEE Communications Surveys and Tutorials, 18(2), pp. 1153-1176.
- Chandola, V., Banerjee, A. and Kumar, V. (2009) 'Anomaly detection: a survey', ACM Computing Surveys, 41(3), pp. 1-58.
- Chawla, N.V., Bowyer, K.W., Hall, L.O. and Kegelmeyer, W.P. (2002) 'SMOTE: synthetic minority over-sampling technique', Journal of Artificial Intelligence Research, 16, pp. 321-357.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: a scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785-794.
- Devlin, J., Chang, M.-W., Lee, K. and Toutanova, K. (2019) 'BERT: pre-training of deep bidirectional transformers for language understanding', Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics, pp. 4171-4186.
- Du, M. and Li, F. (2016) 'Spell: streaming parsing of system event logs', Proceedings of the IEEE 16th International Conference on Data Mining, pp. 859-864.
- Du, M., Li, F., Zheng, G. and Srikumar, V. (2017) 'DeepLog: anomaly detection and diagnosis from system logs through deep learning', Proceedings of the 2017 ACM SIGSAC Conference on Computer and Communications Security, pp. 1285-1298.
- European Parliament and Council (2016) Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016. Brussels: Official Journal of the European Union.

- Goodfellow, I., Bengio, Y. and Courville, A. (2016) *Deep Learning*. Cambridge, MA: MIT Press.
- Guo, H., Yang, J., Liu, J., Bai, J., Wang, X., Li, Z., Zheng, T., Zhang, B., Tian, Z. and Yang, R. (2024) 'LogFormer: a pre-train and tuning pipeline for log anomaly detection', *Proceedings of the 38th AAAI Conference on Artificial Intelligence*, 38(8), pp. 8585-8593.
- Guo, H., Yuan, S. and Wu, X. (2021) 'LogBERT: log anomaly detection via BERT', *Proceedings of the International Joint Conference on Neural Networks (IJCNN)*, pp. 1-8.
- Han, X., Yuan, S. and Trabelsi, M. (2023) 'LogGPT: log anomaly detection via GPT', *Proceedings of the IEEE International Conference on Big Data*, pp. 1117-1124.
- Harris, C.R., Millman, K.J., van der Walt, S.J., Gommers, R., Virtanen, P., Cournapeau, D. and Wieser, E. (2020) 'Array programming with NumPy', *Nature*, 585(7825), pp. 357-362.
- Hastie, T., Tibshirani, R. and Friedman, J. (2009) *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd edn. New York: Springer.
- He, P., Zhu, J., He, S., Li, J. and Lyu, M.R. (2016) 'An evaluation study on log parsing and its use in log mining', *Proceedings of the 46th IEEE/IFIP International Conference on Dependable Systems and Networks*, pp. 654-661.
- He, P., Zhu, J., Zheng, Z. and Lyu, M.R. (2017) 'Drain: an online log parsing approach with fixed depth tree', *Proceedings of the IEEE International Conference on Web Services*, pp. 33-40.
- He, S., Zhu, J., He, P. and Lyu, M.R. (2016) 'Experience report: system log analysis for anomaly detection', *Proceedings of the IEEE 27th International Symposium on Software Reliability Engineering*, pp. 207-218.
- Hochreiter, S. and Schmidhuber, J. (1997) 'Long short-term memory', *Neural Computation*, 9(8), pp. 1735-1780.
- Hunter, J.D. (2007) 'Matplotlib: a 2D graphics environment', *Computing in Science and Engineering*, 9(3), pp. 90-95.
- Information Commissioner's Office (2023) *Guidance on AI and Data Protection*. Wilmslow: Information Commissioner's Office.
- Kingma, D.P. and Ba, J. (2015) 'Adam: a method for stochastic optimization', *Proceedings of the 3rd International Conference on Learning Representations*.
- Landauer, M., Onder, S., Skopik, F. and Wurzenberger, M. (2023) 'Deep learning for anomaly detection in log data: a survey', *Machine Learning with Applications*, 12, p. 100470.
- Le, V.-H. and Zhang, H. (2021) 'Log-based anomaly detection without log parsing', *Proceedings of the 36th IEEE/ACM International Conference on Automated Software Engineering*, pp. 492-504.
- Le, V.-H. and Zhang, H. (2022) 'Log-based anomaly detection with deep learning: how far are we?', *Proceedings of the 44th International Conference on Software Engineering*, pp. 1356-1367.
- LeCun, Y., Bengio, Y. and Hinton, G. (2015) 'Deep learning', *Nature*, 521(7553), pp. 436-444.
- Liang, Y., Zhang, Y., Xiong, H. and Sahoo, R. (2007) 'Failure prediction in IBM BlueGene/L event logs', *Proceedings of the IEEE International Conference on Data Mining*, pp. 583-588.
- Liu, F.T., Ting, K.M. and Zhou, Z.-H. (2008) 'Isolation Forest', *Proceedings of the IEEE International Conference on Data Mining*, pp. 413-422.

- Liu, Y., Ott, M., Goyal, N., Du, J., Joshi, M., Chen, D., Levy, O., Lewis, M., Zettlemoyer, L. and Stoyanov, V. (2019) RoBERTa: a robustly optimised BERT pretraining approach. arXiv preprint arXiv:1907.11692.
- LogHub (2023) LogHub: a large collection of system log datasets. Available at: <https://github.com/logpai/loghub> (Accessed: 12 April 2026).
- Lou, J.G., Fu, Q., Yang, S., Xu, Y. and Li, J. (2010) 'Mining invariants from console logs for system problem detection', Proceedings of the USENIX Annual Technical Conference, pp. 231-244.
- Lundberg, S.M. and Lee, S.-I. (2017) 'A unified approach to interpreting model predictions', Advances in Neural Information Processing Systems, 30, pp. 4768-4777.
- Majors, C., Fong-Jones, L. and Miranda, G. (2022) Observability Engineering: Achieving Production Excellence. Sebastopol, CA: O'Reilly Media.
- Meng, W., Liu, Y., Zhu, Y., Zhang, S., Pei, D., Liu, Y., Chen, Y., Zhang, R., Tao, S., Sun, P. and Zhou, R. (2019) 'LogAnomaly: unsupervised detection of sequential and quantitative anomalies in unstructured logs', Proceedings of the 28th International Joint Conference on Artificial Intelligence, pp. 4739-4745.
- Mitchell, T.M. (1997) Machine Learning. New York: McGraw-Hill.
- Murphy, K.P. (2012) Machine Learning: A Probabilistic Perspective. Cambridge, MA: MIT Press.
- Nedelkoski, S., Bogatinovski, J., Acker, A., Cardoso, J. and Kao, O. (2020) 'Self-supervised log parsing', Proceedings of the European Conference on Machine Learning and Principles and Practice of Knowledge Discovery in Databases, pp. 122-138.
- Oliner, A. and Stearley, J. (2007) 'What supercomputers say: a study of five system logs', Proceedings of the 37th IEEE/IFIP International Conference on Dependable Systems and Networks, pp. 575-584.
- Pang, G., Shen, C., Cao, L. and Hengel, A.V.D. (2021) 'Deep learning for anomaly detection: a review', ACM Computing Surveys, 54(2), pp. 1-38.
- Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., Killeen, T., Lin, Z., Gimelshein, N., Antiga, L. and Desmaison, A. (2019) 'PyTorch: an imperative style, high-performance deep learning library', Advances in Neural Information Processing Systems, 32, pp. 8024-8035.
- Patel, D. (2026) LLM-enhanced log anomaly detection: a comprehensive benchmark of large language models for automated system diagnostics. arXiv preprint arXiv:2604.12218.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V. and Vanderplas, J. (2011) 'Scikit-learn: machine learning in Python', Journal of Machine Learning Research, 12, pp. 2825-2830.
- Powers, D.M.W. (2011) 'Evaluation: from precision, recall and F-measure to ROC, informedness, markedness and correlation', Journal of Machine Learning Technologies, 2(1), pp. 37-63.
- Provost, F. and Fawcett, T. (2013) Data Science for Business. Sebastopol, CA: O'Reilly Media.
- Russell, S. and Norvig, P. (2020) Artificial Intelligence: A Modern Approach. 4th edn. Hoboken, NJ: Pearson.
- Saito, T. and Rehmsmeier, M. (2015) 'The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets', PLoS ONE, 10(3), e0118432.
- Sanh, V., Debut, L., Chaumond, J. and Wolf, T. (2019) DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter. arXiv preprint arXiv:1910.01108.

- Sun, C., Qiu, X., Xu, Y. and Huang, X. (2019) 'How to fine-tune BERT for text classification?', China National Conference on Chinese Computational Linguistics, pp. 194-206.
- Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A.N., Kaiser, L. and Polosukhin, I. (2017) 'Attention is all you need', Advances in Neural Information Processing Systems, 30, pp. 5998-6008.
- Virtanen, P., Gommers, R., Oliphant, T.E., Haberland, M., Reddy, T., Cournapeau, D., Burovski, E., Peterson, P., Weckesser, W., Bright, J. and van der Walt, S.J. (2020) 'SciPy 1.0: fundamental algorithms for scientific computing in Python', Nature Methods, 17(3), pp. 261-272.
- Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M. and Davison, J. (2020) 'Transformers: state-of-the-art natural language processing', Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing, pp. 38-45.
- Xu, W., Huang, L., Fox, A., Patterson, D. and Jordan, M.I. (2009) 'Detecting large-scale system problems by mining console logs', Proceedings of the 22nd ACM Symposium on Operating Systems Principles, pp. 117-132.
- Yang, L., Chen, J., Wang, Z., Wang, W., Jiang, J., Dong, X. and Zhang, W. (2021) 'Semi-supervised log-based anomaly detection via probabilistic label estimation', Proceedings of the 43rd IEEE/ACM International Conference on Software Engineering, pp. 1448-1460.
- Yang, Z. and Harris, I.G. (2025) LogLLaMA: transformer-based log anomaly detection with LLaMA. arXiv preprint arXiv:2503.14849.
- Zhang, X., Xu, Y., Lin, Q., Qiao, B., Zhang, H., Dang, Y., Xie, C., Yang, X., Cheng, Q., Li, Z. and Chen, J. (2019) 'Robust log-based anomaly detection on unstable log data', Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering, pp. 807-817.
- Zheng, T., Chen, J., Shang, W. and Zou, Y. (2022) 'BERT-Log: anomaly detection for system logs based on pre-trained language model', Applied Artificial Intelligence, 36(1), p. 2145642.
- Zhu, J., He, S., Liu, J., He, P., Xie, Q., Zheng, Z. and Lyu, M.R. (2019) 'Tools and benchmarks for automated log parsing', Proceedings of the 41st IEEE/ACM International Conference on Software Engineering: Software Engineering in Practice, pp. 121-130.

Appendix A Source-code Listings

The full source code for the project is available in the project repository under `log_anomaly_rails` for the Rails dashboard and `ml_pipeline` for the Python detection pipeline. The two listings below are the most informative excerpts.

A.1 Drain parser configuration

```
from drain3 import TemplateMiner
from drain3.template_miner_config import TemplateMinerConfig

cfg = TemplateMinerConfig()
cfg.load("drain3.ini")
cfg.profilng_enabled = False
cfg.drain_sim_th = 0.4
cfg.drain_max_depth = 4
miner = TemplateMiner(config=cfg)

for line in open("BGL.log"):
    result = miner.add_log_message(line.strip())
    template_id = result["cluster_id"]
```

A.2 BERT-Log fine-tune entry point

```
from transformers import BertTokenizerFast, BertForSequenceClassification, Trainer

tok = BertTokenizerFast.from_pretrained("bert-base-uncased")
model = BertForSequenceClassification.from_pretrained(
    "bert-base-uncased", num_labels=2)

args = TrainingArguments(
    output_dir="saved_models/bert_log_hf",
    num_train_epochs=10,
    per_device_train_batch_size=16,
    learning_rate=2e-5,
    weight_decay=0.01,
    evaluation_strategy="epoch")

trainer = Trainer(model=model, args=args,
                  train_dataset=train_ds, eval_dataset=val_ds,
                  tokenizer=tok, data_collator=collator)
trainer.train()
```

Appendix B Extended Results Tables

B.1 Per-model confusion counts on the BGL test partition

Model	TP	FP	TN	FN
Logistic Regression	11	7	18	4
Random Forest	11	0	25	4
LogBERT	13	2	23	2
LogGPT	14	2	23	1
PLELog	14	1	24	1
LogFormer	14	1	24	1
BERT-Log	14	1	24	1

B.2 Training time and model size

Model	Train time (min)	Parameters (M)	Disk size (MB)
Logistic Regression	0.4	0.0001	0.1
Random Forest	1.8	12.0	74
LogBERT	52	110	440
LogGPT	78	124	498
PLELog	24	6	23
LogFormer	46	110	440
BERT-Log	38	110	440